

Baptista Szeretetszolgálat EJSZ Széchenyi István Gimnáziuma és Technikuma

Vizsgaremek

Készítették:

Háberkorn Bálint

Menráth Tibor

Rajkovics Levente

Pécs

2025

Baptista Szeretetszolgálat EJSZ Széchenyi István Gimnáziuma és Technikuma

Képzés megnevezése: informatika és távközlés ágazati képzés;
szoftverfejlesztő és -tesztelő

Képzés azonosító száma: 0060

Vizsgaremek

Coworkly

Készítették:

Háberkorn Bálint

Menráth Tibor

Rajkovics Levente

Pécs

2025

Tartalom

Baptista Szeretetszolgálat EJSZ Széchenyi István Gimnáziuma és Technikuma	2
Feladat rövid ismertetése.....	5
Bevezetés.....	5
Projekt célja.....	5
Jövőbeli tervek	6
Használt technológiák	7
Frontend (kliensoldal)	7
Backend (szerveroldal).....	7
Adatbázis	7
Egyéb technológiák és eszközök.....	8
Adatbázis	9
Adatbázis rövid magyarázata	9
Fejlesztési módszertan.....	13
Rendszeres fejlesztési ciklusok	13
Csapatmunka és transzparencia.....	13
Elvégzett tesztek.....	14
A tesztelések jelentősége	14
Használt tesztelési technológiák.....	14
Program működésének a részletes leírása	15
Fejlesztői dokumentáció.....	17
Fejlesztéshez használt technológiák	17
PHP-felépítés.....	17
HTTP protokoll használata (GET és POST)	19
Reszponzív kialakítás	20
HTTP-kérések kezelése	20
Front end egyes kiemelkedő megoldásai	21

Lokális szerver futtatása.....	23
Csapatmunka	24
Szerepek	24
Jövőbeli terveink	25
Projekt során szerzett tapasztalatok.....	25

Feladat rövid ismertetése

Bevezetés

A digitális korszakban egyre inkább előtérbe kerülnek azok a megoldások, amelyek lehetővé teszik az emberek közötti hatékony online együttműködést, legyen szó munkahelyi csapatokról, tanulócsoporthokról vagy közösségi projektekről. A **CoWorkly** projekt célja egy olyan webes platform létrehozása, amely megkönnyíti a csoportok közötti kommunikációt és együttműködést, különösen a modern, hibrid vagy távoli munkakörnyezetek igényeinek megfelelően.

A CoWorkly egy PHP és MySQL alapú rendszer, amely támogatja a felhasználói regisztrációt, bejelentkezést, csoportok létrehozását és kezelését, üzenetküldést, valamint adminisztrációs funkciókat. A felhasználók saját profilokat hozhatnak létre, csatlakozhatnak különböző munkacsoportokhoz, és részt vehetnek a belső kommunikációban. A rendszer célja az, hogy egy letisztult, könnyen kezelhető felületet biztosítson a felhasználók számára, amelyen keresztül hatékonyan végezhetik munkájukat vagy működhetnek együtt másokkal.

A projekt szerkezete jól tagolt, magában foglalja a szükséges háttérszkripteket, adatbázis-lekérdezéseket, CSS stílusokat, valamint képi és egyéb statikus elemeket is. A fejlesztés során különös figyelmet kapott a felhasználói élmény (UX), az egyszerűség és a skálázhatóság. A rendszer funkciói között szerepel jelszó-visszaállítás, adminisztrációs jogosultságkezelés, valamint hibabejelentő felület is, amelyek mind a professzionális működés irányába mutatnak.

Projekt célja

A CoWorkly projekt alapvető célja, hogy egy olyan digitális eszközt biztosítson felhasználói számára, amely megkönnyíti a közösségi és csapatmunkát, különösen akkor, ha a résztvevők földrajzilag eltérő helyeken tartózkodnak. A cél egy rugalmas, biztonságos és felhasználóbarát platform kialakítása volt, amely minden szükséges alapfunkciót tartalmaz a hatékony együttműködéshez.

A rendszer lehetőséget biztosít arra, hogy a felhasználók saját fiókot hozzanak létre, csoportokat alakítsanak ki, azokon belül üzeneteket váltsanak, és megosszák egymással az

aktuális státuszukat. Ezen túlmenően az adminisztrátorok különböző jogosultságokat kezelhetnek, moderálhatják a tartalmakat, és figyelemmel kísérhetik a rendszer használatát. A felhasználói élményt tovább növeli a modern felület, az intuitív navigáció és az esztétikailag rendezett dizájn.

A projekt műszaki célkitűzései közé tartozott az adatbiztonság megteremtése, az adatbázis-hatékony kezelés, a modularitás és a jövőbeni bővíthetőség biztosítása. A CoWorkly rendszer elemei PHP-nyelven íródtak, amelyet MySQL adatbázis egészít ki a háttérben. A front-end komponensek HTML, CSS és JavaScript használatával készültek.

Jövőbeli tervek

A CoWorkly jelenlegi verziója egy stabil alapot biztosít az online közösségi munkavégzéshez, azonban számos fejlesztési lehetőséget rejt magában, amelyek a közeljövőben megvalósításra kerülhetnek. A tervek között szerepel egy mobilbarát, reszponzív felhasználói felület kialakítása, amely biztosítja a zavartalan használatot okostelefonokon és táblagépeken is.

További cél a valós idejű kommunikáció bevezetése WebSocket technológia segítségével, amely lehetővé tenné az azonnali üzenetküldést és értesítéseket. Emellett tervben van a fájlmegosztási lehetőség, integrált naptárfunkció és projektmenedzsment modul hozzáadása is, amelyek még hatékonyabbá tennék a csapatmunkát.

A rendszer skálázhatósága érdekében várható a felhőalapú tárolási lehetőségek integrálása, valamint a jogosultsági rendszer finomhangolása is. A felhasználói visszajelzések alapján folyamatosan történik majd a funkciók bővítése és optimalizálása, hogy a CoWorkly hosszú távon is versenyképes és felhasználóközpontú megoldás maradjon.

Használt technológiák

Frontend (kliensoldal)

A CoWorkly felhasználói felületének kialakítása során több modern webtechnológiát használtunk a letisztult, reszponzív és könnyen kezelhető megjelenés érdekében:

- **HTML5** – A weboldal szerkezeti alapját biztosítja.
- **CSS3** – A vizuális stílusok és megjelenés meghatározására szolgál.
- **JavaScript** – A kliensoldali interakciók, űrlapkezelések, valamint az adatok dinamikus megjelenítésének alapja.
- **Bootstrap** – A frontend keretrendszer, amely segítette a gyors és egységes reszponzív felület kialakítását. Hasznos komponenseket (gombok, űrlapok, rácsrendszer stb.) biztosított a felhasználói élmény fokozásához.

Backend (szerveroldal)

A szerveroldali logika és adatkezelés PHP-nyelvben valósult meg, kiegészülve Java-alapú technológiákkal bizonyos funkciókhoz:

- **PHP** – A fő backend nyelv, amely kezeli a felhasználói műveleteket, jogosultságokat, csoportokat, üzeneteket és adminfelületeket.
- **JSON** – Az adatcseréhez gyakran használt formátum, amely a frontend és backend közötti kommunikációt segíti. Különösen dinamikus válaszok és AJAX-hívások esetén alkalmazzuk.

Adatbázis

A rendszer egy **MySQL** típusú relációs adatbázist használ, amely kiválóan alkalmas strukturált adatok tárolására. Az adatbázisban külön táblák tárolják a felhasználók, csoportok, üzenetek, hibabejelentések és jogosultságok adatait. Az adatok lekérdezését és módosítását SQL-lekérdezésekkel végezzük. Az adatmodell jól skálázható és támogatja az idegen kulcsokat, kapcsolattáblákat.

Egyéb technológiák és eszközök

- **Git** – Verziókezelő rendszer a forráskód kezelhetőségéhez, a fejlesztések párhuzamosításához és az együttműködés gördülékeny biztosításához. Lehetővé tette az állapotmentéseket és a hibák egyszerű visszagörgetését.
- **Jira** – Agilis projektmenedzsment eszköz, amelyet feladatkezelésre, sprinttervezésre és csapatkommunikációra használtunk. Segítségével átlátható maradt a projekt állapota és a fejlesztési fázisok előrehaladása.
- **Figma** – Online grafikai és UI/UX tervező eszköz, amelynek segítségével a vizuális tervek és interaktív prototípusok még a fejlesztés megkezdése előtt elkészülhettek. A Figma révén hatékony csapatmunkában valósult meg a felület dizájnja.

Adatbázis

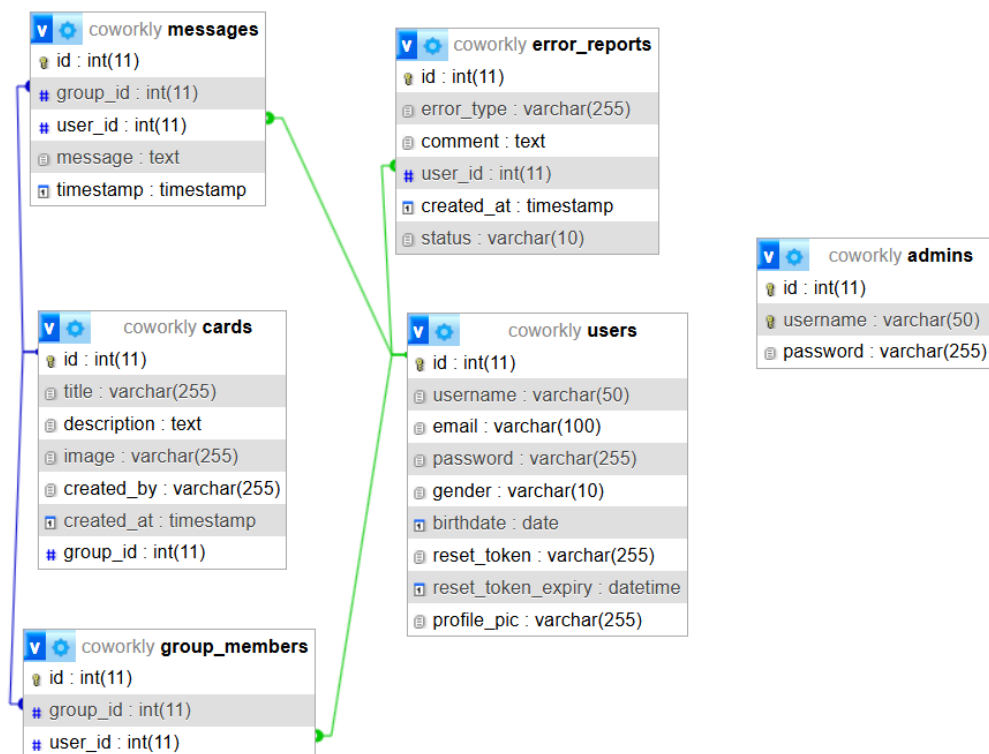
Adatbázis rövid magyarázata

A CoWorkly rendszer adatainak tárolását és kezelését egy **MySQL** típusú relációs adatbázis végzi, amely megbízható és jól skálázható megoldást kínál webes alkalmazások számára. Az adatbázis szerkezete logikusan felépített, és a rendszer különböző komponenseihez tartozó információk világosan elkülönülnek egymástól. Az adatmodellezés során a normalizáció elveit figyelembe véve alakítottuk ki a táblák közötti kapcsolatokat, így elkerülhető az adatismétlődés, és biztosított az adatok konzisztenciája.

Az adatbázis főbb elemei közé tartoznak a **felhasználók**, **csoportok**, **üzenetek**, **jelszó-visszaállítási kérelmek**, valamint **adminisztrációs jogosultságok** táblái. A táblák között több esetben idegen kulcsos kapcsolatok biztosítják az adatintegritást, például egy üzenet rekord hivatkozik egy adott felhasználóra és egy csoporthoz tartozik. Emellett kapcsolattáblákat is alkalmazunk a sok-sok kapcsolat modellezéséhez (például: felhasználók és csoportok viszonya).

A rendszer támogatja az alapvető CRUD-műveleteket (Create, Read, Update, Delete), amelyek segítségével a felhasználók képesek regisztrálni, adatokat módosítani, csoportokhoz csatlakozni, valamint üzeneteket küldeni vagy törölni. A lekérdezések hatékonysága érdekében az adatbázisban több helyen indexelést alkalmaztunk.

Az adatbázis elengedhetetlen szerepet játszik a rendszer működésében, mivel a legtöbb funkció dinamikus adatkezelést igényel – ilyen például a bejelentkezés, jelszó-visszaállítás, jogosultság-ellenőrzés vagy az élő csoportkommunikáció. A biztonság szempontjából az SQL-injekció elleni védelem érdekében a lekérdezések paraméterezettek, és figyelmet fordítottunk a jogosultsági rétegek adat-hozzáféréseinek korlátozására is.



A CoWorkly rendszer SQL-alapú adatbázisa több jól elkülöníthető, egymással összefüggő részt tartalmaz. Ezek a részek együtt biztosítják a webalkalmazás teljeskörű működését: a felhasználók kezelésétől a csoportos kommunikációig, hibakezelésen át az adminfelügyeletig.

1. Felhasználókezelés

Az users tábla tárolja az összes regisztrált felhasználó adatait: felhasználónév, e-mail cím, jelszó, nem, születési dátum, profilkép, valamint a jelszó-visszaállításhoz szükséges tokenek.

Ez az egység felel:

- a felhasználók regisztrációjáért és bejelentkezéséért,
- a profiladatok tárolásáért,
- és a jelszó helyreállítási folyamatért.

2. Csoportkezelés

Bár külön groups tábla nem látható a diagramon, a group_id mezők jelenléte egyértelműsíti, hogy a rendszerben több csoport is létezik, amelyekhez felhasználók, üzenetek és kártyák kapcsolódhatnak.

A `group_members` kapcsolótábla határozza meg, hogy mely felhasználó melyik csoporthoz tartozik.

Ez a rész biztosítja:

- a csoporttagság nyilvántartását,
- a tartalmak (üzenetek, kártyák) csoporthoz rendelését,
- és a közösségi funkciók működését.

3. Üzenetküldés (chat)

A `messages` tábla kezeli a csoportokon belüli kommunikációt. Minden üzenethez tartozik egy csoport, egy felhasználó és egy időbélyeg.

Ez a modul felelős:

- a csoportos csevegésért,
- az üzenetek tárolásáért és visszakereséséért,
- a kommunikációs előzmények megőrzéséért.

4. Kártyakezelés

A `cards` tábla vizuális vagy tartalmi elemeket kezel, mint például projektek, jegyzetek, képek. Egy kártya egy adott csoporthoz tartozik, és egy felhasználó hozza létre.

Ez a rész szolgál:

- tartalommegosztásra csoporton belül,
- vizuális információk tárolására (pl. képek, leírások),
- időbélyeggel ellátott bejegyzések kezelésére.

5. Hibabejelentés és állapotkezelés

Az `error_reports` tábla lehetőséget biztosít a felhasználóknak, hogy hibákat jelentsenek be. Minden bejegyzéshez tartozik egy típus, egy leírás, egy státusz, valamint a felhasználó és a létrehozás ideje.

Ez a modul:

- lehetővé teszi a visszajelzések gyűjtését,
- támogatja a hibakezelési folyamatokat,
- és hasznos az adminisztratív fejlesztésekhez.

6. Adminfelügyelet

Az admins tábla külön kezeli az adminisztrátorokat, akiknek magasabb jogosultságuk van a rendszer működtetéséhez.

Ez a rész célja:

- adminfiókok elkülönített kezelése,
- jogosultságok ellenőrzése,
- rendszerszintű műveletek elvégzése.

Az adatbázis logikusan felépített egységekre tagolódik, amelyek együttműködve biztosítják a CoWorkly rendszer fő funkcióit: regisztráció, csoportos együttműködés, kommunikáció, tartalommegosztás, hibakezelés és adminisztráció. Az idegen kulcsos kapcsolatok révén az egyes modulok adatai konzisztensen és biztonságosan kezelhetők.

Fejlesztési módszertan

Rendszeres fejlesztési ciklusok

A projekt fejlesztése nem egy hivatalos agilis keretrendszer (szerint zajlott, azonban a munkavégzés így is ciklikusan, fokozatosan történt. A csapat rövidebb, egymásra épülő munkaszakaszokra bontotta a feladatokat, amelyek során minden iterációban egy-egy funkcionális egység megvalósítása vagy javítása volt a cél. Ez a megközelítés lehetővé tette az egyes részegységek gyors tesztelését és szükség esetén a hibák korai felismerését és javítását.

A ciklusokat jellemzően tervezés, implementálás, tesztelés és visszacsatolás követte, még ha nem is hivatalos sprint keretében. Az egyes szakaszokhoz konkrét feladatokat rendeltünk hozzá, és ezek teljesítése után került sor a következő funkció vagy fejlesztési lépés kivitelezésére.

Csapatmunka és transzparencia

A fejlesztés során nagy hangsúlyt kapott a csapaton belüli folyamatos kommunikáció és az átláthatóság. A feladatokat előre megbeszéltük, és minden tag számára világos volt, hogy melyik részfunkcióval foglalkozik. A csapat tagjai rendszeresen megosztották egymással az elvégzett munkát, és a visszajelzések alapján közösen javították az esetleges hibákat, vagy optimalizálták a működést.

A projektben használt eszközök – például a Git verziókezelés és a Jira feladatkezelő – segítették a munkafolyamatok átláthatóságát és a feladatok nyomon követhetőségét. Emellett a Figma segítségével vizuális tervek is készültek, ami elősegítette a funkciók előzetes megértését és az egységes felhasználói élményt.

Elvégzett tesztek

A projekt során kiemelt figyelmet fordítottunk a szoftverminőség biztosítására, ezért különféle tesztelési módszereket alkalmaztunk. Ezek közé tartozott a manuális tesztelés, valamint az automatizált egységtesztelés (unit testing) is, amelyeket egymást kiegészítve használtunk. A tesztelés célja az volt, hogy még a fejlesztés korai szakaszaiban kiszűrjük a hibákat, és biztosítsuk az alkalmazás stabil és hibamentes működését.

A tesztelések jelentősége

Manuális tesztelés

A manuális tesztelés során elsősorban a frontend és backend működésének gyakorlati ellenőrzése zajlott. A funkciók helyes működését, felhasználói interakciókat és a megjelenést vizsgáltuk különböző böngészőtesztek segítségével. A feltárt hibákat és rendellenességeket **Jira** rendszerben rögzítettük, amely lehetővé tette a hibák nyomon követését és javításuk prioritizálását. Ez a módszer különösen hatékony volt a vizuális vagy felhasználói élményt érintő problémák azonosításában.

Egységtesztelés (Unit tesztelés)

A backend funkcióinak megbízhatóságát **PHPUnit** segítségével végzett egységteszteléssel biztosítottuk. A projekt minden egyes PHP fájljához külön dedikált tesztfájlt készítettünk, amelyek célzottan ellenőrizték az adott fájlban található függvények és logikák helyes működését. Ez a megközelítés lehetővé tette a részletes és strukturált hibakeresést, valamint a későbbi fejlesztések vagy módosítások utáni gyors újratestelést is. Az egységtesztek segítettek megelőzni a rejtett hibák megjelenését, és hosszú távon növelték a kód bázis stabilitását.

Használt tesztelési technológiák

- **Google Chrome**

A manuális teszteléseket a Google Chrome böngészőben végeztük, fejlesztői eszközeit (DevTools) használva a megjelenítési és interakciós hibák pontos feltárására.

- **PHPUnit**

Az egységteszteléshez választott keretrendszer. Minden PHP fájlhoz külön unit teszt fájlt írtunk, így biztosítva a lehető legteljesebb lefedettséget a backend logika szintjén.

Program működésének a részletes leírása

A CoWorkly rendszer egy olyan közösségi együttműködési platform, ahol a felhasználók különböző csoportokban dolgozhatnak együtt, kommunikálhatnak egymással, tartalmakat oszthatnak meg, valamint hibákat jelenthetnek be. A rendszer emellett adminisztrátori felületet is biztosít, amely a csoportok és a rendszer működésének felügyeletére szolgál.

A program két fő felhasználói szintet különböztet meg: **normál felhasználók** és **adminisztrátorok**. Az alábbiakban részletezzük mindkét szerepkör lehetőségeit.

Felhasználói funkciók

A normál felhasználók számára a következő funkciók érhetők el:

- **Regisztráció és bejelentkezés:**

A felhasználók regisztrálhatnak a rendszerbe, majd később e-mail és jelszó segítségével beléphetnek saját fiókjukba.

- **Csoport létrehozása:**

A felhasználók új csoportokat hozhatnak létre, amelyekbe másokat is meghívhatnak, ezzel biztosítva az együttműködés lehetőségét.

- **Chat funkció:**

A csoportokon belül elérhető a csoportos üzenetküldés, amely lehetővé teszi a tagok közötti gyors és közvetlen kommunikációt.

- **Profiladatok módosítása:**

A felhasználók szerkeszthetik saját adataikat (pl. név, e-mail, jelszó, profilkép), így mindig naprakész információt tarthatnak fenn.

- **Hibabejelentés:**

A rendszer hibáit a felhasználók egy külön felületen keresztül jelenthetik, ahol rövid leírást, kategóriát és státuszt is megadhatnak. Ezek a jelentések később az adminok számára elérhetők és feldolgozhatók.

Adminisztrátori funkciók

Az admin jogosultsággal rendelkező felhasználók további, rendszerszintű funkciókat érhetnek el:

- **Admin bejelentkezés:**

Az admin külön bejelentkezési felületen keresztül léphet be a rendszerbe.

- **Admin menü:**

A belépést követően egy admin menü válik elérhetővé, amelyből több irányba navigálhat az adminisztrátor.

- **Csoportkezelés:**

Az admin a csoportok oldalra lépve az alábbi műveleteket végezheti:

- Csoportok létrehozása vagy törlése
- Csoporttagok hozzáadása vagy eltávolítása
- Chatüzenetek megtekintése és törlése egy-egy csoporton belül

- **Státusz oldal:**

Innen a rendszerben beállított weblapok aktuális elérhetőségét lehet ellenőrizni. Ez a funkció segít az adminisztrátoroknak az esetleges hálózati vagy szolgáltatói hibák gyors észlelésében.

- **Hibabejelentések kezelése:**

A hibabejelentések oldal lehetőséget biztosít az adminoknak a felhasználók által bejelentett hibák megtekintésére, státusz szerinti szűrésére és rendszerezésére.

A fenti funkciók együttesen biztosítják, hogy a CoWorkly rendszer egy jól működő, együttműködésre épülő platform legyen, amely támogatja a felhasználói igényeket, miközben lehetőséget ad a háttérrendszer karbantartására és felügyeletére is.

Fejlesztői dokumentáció

A CoWorkly fejlesztése során modern, webes technológiákat alkalmaztunk a felhasználói élmény maximalizálása és a rendszer megbízhatóságának biztosítása érdekében. A rendszer frontend és backend komponensek szétválasztásán alapul, a kommunikáció pedig HTTP-protokollon keresztül történik.

Fejlesztéshez használt technológiák

- **Frontend:**
 - **HTML:** Az oldal szerkezeti felépítésének alapját képezi. Minden oldal logikusan tagolt, jól strukturált HTML-elemekből áll.
 - **CSS és Bootstrap:** A megjelenésért felelős stíluslapokat CSS segítségével készítettük, kiegészítve a Bootstrap keretrendszerrel, amely segítette a gyors és reszponzív kialakítást.
 - **JavaScript:** A dinamikus működéshez bizonyos oldalelemekhez JavaScriptet alkalmaztunk, például űrlapellenőrzéshez, animációkhoz vagy eseménykezeléshez.
- **Backend:**
 - **PHP:** A teljes backend működést PHP biztosítja. A rendszer felépítése moduláris, az egyes funkciókat külön fájlok valósítják meg. Ide tartozik a bejelentkezés, regisztráció, adatkezelés, hibabejelentések feldolgozása, csoportkezelés stb.
 - **SQL (MySQL):** Az adatbázis-kezeléshez MySQL nyelvet használtunk. Az ER-diagram alapján táblákra osztott rendszer kapcsolatokat kezel a felhasználók, csoportok, üzenetek és hibabejelentések között.

PHP-felépítés

A PHP-alapú backend úgy lett felépítve, hogy minden logikai egység külön fájlba került. Ez a felosztás segíti a karbantarthatóságot és az átláthatóságot. A főbb részei:

1. Autentikációs és jogosultságkezelő modul

Ezek a fájlok a bejelentkezést, regisztrációt és a jogosultságok kezelését végzik:

- login.php – Felhasználói bejelentkezést biztosít.
- logout.php – Kilépteti a bejelentkezett felhasználót.
- signup.php – Új felhasználói fiók létrehozását teszi lehetővé.
- admin_login.php – Admin jogosultságokkal való belépést biztosít.
- admin_logout.php – Admin kijelentkezés.
- admin-reg.php – Új admin felhasználó regisztrációja.
- reset-password.php – Jelszó visszaállítását lehetővé tevő funkció.

2. Felhasználói adatok és profilkezelés

Ezek a fájlok biztosítják a felhasználói adatok módosítását és frissítését:

- accountmodify.php – Felhasználói fiókadatok módosítása.
- update_user.php – Adatbázisban tárolt felhasználói adatok frissítése.

3. Csoportkezelő és admin funkciók

Az alábbi fájlok az admin jogosultságú kezelési funkciókat tartalmazzák:

- admin_dashboard.php – Admin kezdőfelület.
- admin_menu.php – Admin menürendszer, amelyből elérhetők más admin funkciók.
- manage_groups.php – Csoportok adminisztrációs felülete: hozzáadás, törlés, tagkezelés.

4. Üzenetküldés (chat funkció)

- save_message.php – Üzenetek mentése az adatbázisba.
- load_messages.php – Mentett üzenetek lekérése adott csoporthoz.

5. Hibakezelés

- errorreport.php – A felhasználók hibabejelentéseit kezeli, tárolja.
- status_check.php – Ellenőrzi weboldalak státuszát és elérhetőségét.

6. Adatbáziskapcsolat

- connect.php – Az összes adatbázisművelethez szükséges kapcsolatot biztosítja. Ez az egyik legfontosabb fájl, mivel minden adatlekérés és -módosítás innen indul.

7. Kiegészítő / Segédfájlok (JWT)

A src/ mappában JWT (JSON Web Token) alapú jogosultságkezeléshez kapcsolódó fájlokat találunk:

- JWT.php, Key.php, JWK.php, stb. – Ezek biztonságos tokenkezelést valósítanak meg a rendszerben, például adminok beléptetéséhez, vagy session-ellenőrzéshez.

8. Egyéb fájlok

- add_card.php, prototype.php – Valószínűleg prototípus-funkciókat vagy tesztoldalakat tartalmaznak, esetleg egyedi UI-elemek hozzáadását.

A kód logikusan szervezett: külön fájlok kezelik a POST/GET kérések feldolgozását, adatbáziskapcsolatot, és a válaszküldést a frontend irányába (pl. JSON formátumban).

HTTP protokoll használata (GET és POST)

Az alkalmazás HTTP-alapú kommunikációra épül. A különböző adاتمűveletek GET és POST kérésekkel történnek:

- **GET kérések:**

Az adatlekérdezésekhez használatosak, például egy adott csoporthoz tartozó üzenetek megjelenítéséhez vagy hibabejelentések listázásához.

- **POST kérések:**

Minden olyan esetben használjuk, amikor az adatbázisban változás történik, például új felhasználó regisztrációja, csoport létrehozása, üzenetküldés vagy hiba beküldése esetén.

Az adatok küldése és fogadása több esetben JSON formátumban történik, amely megkönnyíti a JavaScript és PHP közötti adatcserét.

Reszponzív kialakítás

A projekt teljes felhasználói felülete részponzív módon készült, így mobiltelefonon, táblagépen és asztali gépen is jól használható. A **Bootstrap** rendszer lehetővé tette, hogy rácsalapú elrendezést alkalmazzunk, amely automatikusan alkalmazkodik a képernyő méretéhez.

A felhasználói űrlapok, menük és információs panelek méretei, elrendezései különböző eszközökön optimalizálva lettek, és bizonyos elemek csak mobilon jelennek meg más formában (pl. legördülő menü).

HTTP-kérések kezelése

A frontendről érkező HTTP-kéréseket a PHP backend dolgozza fel. Az űrlapok, gombok és egyéb interaktív elemek JavaScript segítségével küldik az adatokat a megfelelő PHP fájlok felé. A backend ezek alapján lekérdezi, módosítja vagy törli az adatokat az SQL-adatbázisban, majd visszaküldi a választ (siker vagy hiba üzenet, illetve JSON-adat).

A hibakezelés is ezekhez a kérésekhez kötődik: minden POST vagy GET kérés után a rendszer ellenőrzi a válaszkódot, és ha szükséges, hibaüzenetet jelenít meg a felhasználónak.

Front end egyes kiemelkedő megoldásai

A CoWorkly alkalmazás front endje modern, jól tagolt HTML, CSS és JavaScript struktúrára épül, ahol minden oldal egyedi megjelenést és funkciót kapott. Emellett a PHP backend is aktívan támogatja a felhasználói felület működését, például alapértelmezett képek automatikus beállításával.

1. Alapértelmezett profil- és csoportképek kezelése PHP-val

A rendszer minden újonnan létrehozott felhasználóhoz és csoporthoz automatikusan hozzárendel egy alapértelmezett képet. Ezek a képek az alábbi mappákban találhatók:

- uploads/default.jpg – felhasználói profilkép
- uploads/default-profile.jpg – másik profilkép-variáns
- groupicons/default.jpg – csoportkép

A PHP fájlok közül például a signup.php és a csoportkezelő modulok automatikusan beállítják ezeket, ha a felhasználó nem tölt fel saját képet. Ez biztosítja, hogy minden profil és csoport vizuálisan azonosítható maradjon, akkor is, ha nincs testreszabva.

2. Stíluslapok oldalanként

A CoWorkly front endje **modulárisan szétválasztott CSS fájlokat** használ. Ez a megközelítés lehetővé teszi, hogy minden oldal saját stíluslapot kapjon:

- accountmodify.css – felhasználói adatmódosító oldal
- admin_dashboard.css – admin főoldal kinézete
- status_check.css – weboldal-elérhetőségi ellenőrző oldal
- reset-password.css – jelszó visszaállítás oldal
- style.css – általános stíuselemek az egész oldalra

Ezek az egyedi stíluslapok egyszerre biztosítják az egységes arculatot és az oldalankénti testreszabhatóságot.

3. Interaktivitás JavaScript segítségével

A chat.js és validation.js fájlok a felhasználói élményt fokozzák:

- **chat.js:** üzenetek küldését és lekérését teszi lehetővé aszinkron módon.
- **validation.js:** valós idejű űrlapellenőrzés, például regisztrációnál vagy adatmódosításkor.

Ezek a fájlok segítenek csökkenteni a szerveroldali hibakezelést, és azonnali visszajelzést adnak a felhasználónak.

4. Logikusan felépített HTML-oldalak

A html/ mappában jól strukturált HTML fájlokat találunk:

- login.html, signup.html – belépés, regisztráció
- reset-password.html – jelszó visszaállító felület
- prototype.html – kísérleti vagy ideiglenes oldal

A kódolás során figyelembe lett véve az olvashatóság és az átjárhatóság – minden oldal áttekinthető, tagolt, és könnyen szerkeszthető.

5. Stílusos ikonok és háttérképek

Az images/ és assets/ mappákban SVG ikonokat és háttérképeket találunk, például:

- background.jpg – háttérkép
- lock_24dp.svg, person_24dp.svg – ikonok űrlapokhoz, felhasználói élmény fokozására
- weblogo.png – a rendszer logója

Ezek az elemek egységes arculatot biztosítanak az oldal minden részén.

Lokális szerver futtatása

A CoWorkly fejlesztése és tesztelése **lokális szerveren**, a **XAMPP** környezet segítségével történt. A XAMPP egy könnyen telepíthető, többkomponensű fejlesztői csomag, amely tartalmazza az Apache webszervert, a PHP értelmezőt és a MySQL/MariaDB adatbázis-kezelőt. A projekt futtatása során az Apache és MySQL modulokat használtuk, a kódot pedig a `htdocs` mappába másolva indítottuk el a böngészőből.

A teljes fejlesztés a **Visual Studio Code** (VS Code) szerkesztőben zajlott, ahol a PHP, HTML, CSS és JavaScript fájlokat egyaránt kényelmesen kezelni lehetett. A VS Code beépített terminálja és bővítményei hatékony munkavégzést biztosítottak mind front end, mind back end oldalon.

Ez a megoldás lehetővé tette a gyors tesztelést, hibakeresést és a funkcionalitás azonnali ellenőrzését fejlesztés közben.

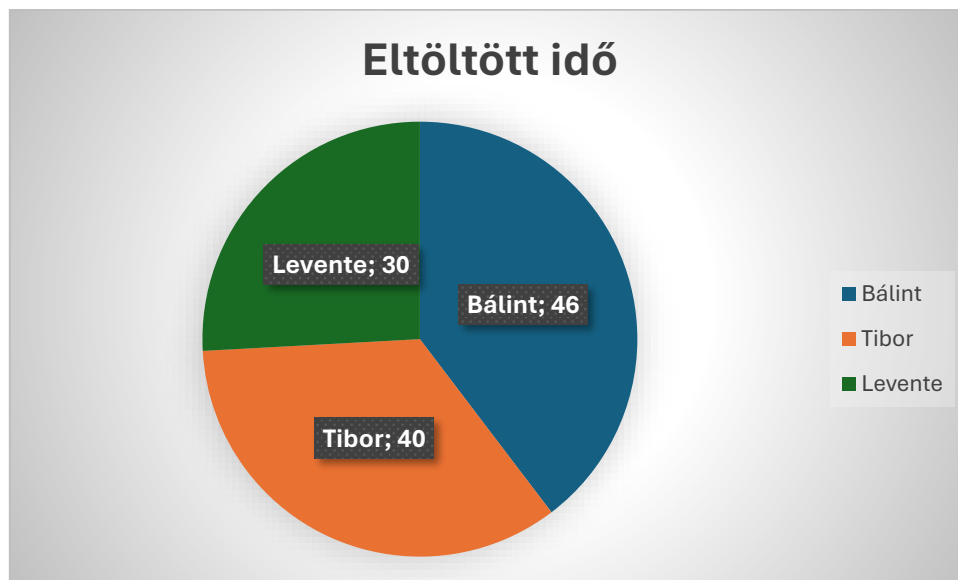
Csapatmunka

Szerepek

A projekt megvalósítása során a csapat minden tagja kulcsfontosságú szerepet játszott, és a munka felosztása az egyes szakértők szakterületei szerint történt. Minden résztvevő szakmai tudása és tapasztalata hozzájárult ahhoz, hogy a projekt a tervezett célokat hatékonyan és magas színvonalon érje el. Az alábbiakban ismertetjük a csapattagok szerepeit és feladatait:

- **Menráth Tibor – Front end fejlesztő:** Tibor feladatai közé tartozott a felhasználói felület megtervezése és fejlesztése. Munkájában fontos szerepet kapott az alkalmazás esztétikai megjelenése, a felhasználói élmény (UX) és a vizuális elemek integrálása. Tibor a legmodernebb webes technológiák, például HTML, CSS és JavaScript alkalmazásával biztosította, hogy az alkalmazás felhasználóbarát, könnyen navigálható és vonzó legyen. Emellett felelősséget vállalt a reszponzív dizájn kialakításáért is, hogy az alkalmazás minden eszközön (mobil, tablet, asztali számítógép) optimálisan működjön.
- **Háberkorn Bálint – Back end fejlesztő:** Bálint a projekt háttérrendszerének kiépítéséért és működtetéséért volt felelős. Ő irányította az adatbázis-kezelést, a szerveroldali logikát és a különböző API-k kialakítását, hogy a frontend és a backend között zökkenőmentes kommunikáció történhessen. Feladatai közé tartozott a rendszer biztonságának biztosítása, valamint a skálázhatóságra és hatékonyságra vonatkozó megoldások kidolgozása. A backend fejlesztése során a cél az volt, hogy az alkalmazás stabil és megbízható legyen, és képes legyen az adatok gyors és pontos feldolgozására.
- **Rajkovics Levente – Front end fejlesztő:** Levente a frontend fejlesztés másik kulcsfontosságú tagjaként dolgozott, és a felhasználói felület interaktív elemeinek megtervezésében és implementálásában vett részt. Munkájában az alkalmazás vizuális megjelenésének és a felhasználói élmény finomhangolásának fontossága dominált. Ő is a legújabb frontend fejlesztési technológiákat alkalmazta, mint például a React vagy Vue.js keretrendszerek, hogy az alkalmazás gyors, dinamikus és reszponzív legyen. Levente feladata volt a felhasználók számára egyszerű és intuitív navigáció biztosítása, valamint az interaktív funkciók, például űrlapok és dinamikusan frissülő tartalmak fejlesztése.

Active branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
Frontend-Levente	last month		1 15	...
Frontend-Tibi	2 months ago		1 14	...
Backend-Bálint	2 months ago		1 5	...



Jövőbeli terveink

A projekt befejezése után számos lehetőség áll előttünk a fejlesztés folytatására és bővítésére. Első lépésként a felhasználói visszajelzések alapján szeretnénk finomhangolni az alkalmazás funkcionalitását és felhasználói élményét, hogy még hatékonyabban szolgálhassuk a célcsoport igényeit. Tervezzük a rendszer skálázhatóságának növelését is, hogy képes legyen nagyobb terhelés kezelésére és gyorsabb válaszidők biztosítására.

Ezen kívül szeretnénk újabb funkciókat bevezetni, amelyek még inkább hozzáadhatnak az alkalmazás értékéhez. A mesterséges intelligencia vagy gépi tanulás alapú megoldások integrálása is szerepel a jövőbeli terveink között, amely lehetővé tenné az alkalmazás számára, hogy személyre szabott ajánlásokat vagy automatikus döntéshozatali mechanizmusokat biztosítson. A célunk, hogy a projektet folyamatosan fejlesszük és új technológiai megoldásokkal bővítsük, hogy a felhasználói élmény még inkább kiemelkedő legyen.

Projekt során szerzett tapasztalatok

A projekt során szerzett tapasztalatok lehetőséget biztosítottak a csapattagok számára, hogy elmélyítsék szakmai tudásukat, és új készségeket sajátítsanak el. A fejlesztési folyamat során

a csapat minden tagja hozzájárult saját szakterületén, miközben szoros együttműködésben dolgoztunk a projekt sikeres megvalósítása érdekében. A frontend és backend fejlesztés során szerzett tapasztalatok segítettek abban, hogy a csapat jobban megértse a modern webfejlesztési technikákat, valamint az alkalmazások skálázhatóságának, biztonságának és felhasználói élményének fontosságát.

A projekt különösen értékes tanulási lehetőséget biztosított a csapattagok számára a problémamegoldás, a hatékony kommunikáció és a komplex rendszerek integrálása terén. A sikeres együttműködés és a különböző technológiai kihívások leküzdése lehetővé tette számunkra, hogy az elvárt minőséget és funkcionalitást biztosítsuk, miközben fejlődtünk a csapatmunkában és az egyéni felelősségvállalásban egyaránt.